

EXERCISE 1 · MID DECK 1 · ANSWER KEY

First Calls and the Converse API

Apply the basics — Bedrock, Converse, system prompts, inference parameters.

6 questions · For trainer use · Distribute after the exercise

HOW TO USE

Walk through each answer with the cohort. The explanation is what matters — many of these have answers that look right but fail for subtle reasons.

Q1 OF 6 · CONCEPT

Amazon Bedrock is described as a *fully managed* service. Which of the following best captures what "fully managed" actually means here?

- A. Bedrock writes your prompts for you so you never have to learn prompt engineering.
- B. Bedrock provisions and operates the model servers; you pay per token and call an API.
- C. Bedrock guarantees the model will never hallucinate because AWS manages quality.
- D. Bedrock is free for the first million tokens of every model.

ANSWER

B.

"Fully managed" refers strictly to infrastructure: AWS runs the servers, you call the API, you pay per token. It does not imply correctness, free usage, or automated prompt engineering. A and C are common misreadings of the marketing copy.

Q2 OF 6 · BUG SPOT

A teammate writes the following code and complains that it throws *ValidationException: model not found*. What is wrong?

```
import boto3

client = boto3.client("bedrock-runtime", region_name="us-east-1")
model_id = "amazon.nova-lite-v1:0"

response = client.converse(
    modelId=model_id,
    messages=[{"role": "user", "content": [{"text": "Hello"}]}],
    inferenceConfig={"temperature": 0.7, "maxTokens": 200},
)
```

ANSWER

Missing the `us.` inference-profile prefix on the model ID.

Newer Bedrock models require a regional inference-profile prefix (`us.`, `eu.`, `apac.`). The model ID should be `"us.amazon.nova-lite-v1:0"`. Without it, Bedrock can't route the request and returns *ValidationException* or *ModelNotFoundException*.

Q3 OF 6 · CONCEPT

What is the **main practical advantage** of using the Converse API instead of per-provider invoke endpoints?

- A. Converse is the only API that supports streaming responses.
- B. Converse encrypts your prompts before sending them to the model.
- C. Converse uses the same request and response shape regardless of which model provider you call.
- D. Converse automatically translates your prompt into the model's native language.

ANSWER

C.

The single biggest practical benefit is provider-agnostic shape. Swap Nova for Claude or Llama by changing only modelId; the rest of your code is unchanged. A is false (streaming works elsewhere too), B is meaningless, D is fiction.

Q4 OF 6 · BUG SPOT

Look at this prompt setup. Why is this an *anti-pattern*, even though it returns a valid response?

```
system_prompt = [{
    "text": "You are a helpful assistant. The user is asking: "
    "What is serverless computing?"
}]

messages = [
    {"role": "user", "content": [{"text": ""}]}
]
```

ANSWER

The user question is buried *inside* the system prompt, and the user message is empty.

The system prompt is meant for stable, behavior-defining instructions. Putting the user's question in there mixes roles, prevents prompt caching of the system prefix (because the system prompt now changes every call), and confuses the model's role boundaries. The user's question belongs in the user message.

Q5 OF 6 · CONCEPT

Which inference parameter would you change to make the model's output **more deterministic and predictable**, e.g. for a classification task?

- A. Increase maxTokens
- B. Decrease temperature toward 0.0
- C. Set temperature to 1.0
- D. Add more text to the system prompt

ANSWER

B.

Temperature controls randomness. 0.0 is the most predictable end; 1.0 is the most varied. For classification, extraction, or anything where you want the same input to produce the same output, push temperature toward 0.

Q6 OF 6 · DESIGN SCENARIO

You're building an internal tool that calls Bedrock to classify customer support tickets into one of five categories.

Pick reasonable values for `temperature` and `maxTokens` and **justify each in one sentence**. Mention one risk if either value is set wrong.

ANSWER

Reasonable: `temperature=0.0` (or 0.1), `maxTokens` around 20-50.

Classification wants determinism — same ticket should always get the same category. Temperature near 0 enforces that. `maxTokens` stays low because the output is a category label, not a paragraph. Risks: high temperature → unstable categorization; high `maxTokens` → either wasted spend if the model rambles, or no harm if you parse only the first word. Bigger risk: forgetting to set `maxTokens` at all — a runaway response can chew through budget.